

Extension=.otf, UprightFont=*-regular, BoldFont=*-bold, ItalicFont=*-italic,
 BoldItalicFont=*.bolditalic texgyretermes texgyreheros texgyrecursor
OPEN-MOPD: DIAGNOSING AND FIXING CAPABIL-
ITY IMBALANCE IN
MULTI-TEACHER ON-POLICY DISTILLATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Reinforcement learning has made single-domain teachers steadily stronger, and integrating their capabilities into one deployable model remains an open problem in LLM post-training. Multi-teacher on-policy distillation (M-OPD)—route each prompt to the expert for its domain, and let that expert supply a dense per-token reward on the student’s own samples—is the route taken in practice, yet the community has no open recipe for it and its mechanism remains unexplored. We build one from scratch on SmoLM3-3B-Base: a three-domain mixed SFT, math / code / instruction-following RL teachers forked from that same start, and M-OPD that routes hard on domain labels. Even with routing error removed, the naive pipeline falls short of its own per-domain oracle, and it does so unevenly across domains. Per-domain measurement locates this capability integration gap in the allocation of optimisation budget, along three independent axes: prompt share is not gradient-token share (IF holds 20.3% of the prompts and contributes 0.99% of the tokens); a flattened token budget is pulled apart again by teacher–student gaps that shrink at different rates; and throughput optimisation leaves the student-dependent part of the reward stale inside a rollout batch. Open-MOPD repairs one axis per mechanism, each first confirmed to change the quantity it claims to control, and closes most of that gap, raising recovery relative to a domain-routed RL oracle from 35.6% to 83.4%. We release the full recipe and training scripts.

1 INTRODUCTION

As reinforcement learning has emerged as a principal method for improving the capabilities of large language models (LLMs), single-domain teachers have become steadily stronger, and integrating their capabilities into one deployable model remains an open problem in modern LLM post-training. The most promising route, and the one actually taken in practice, is multi-teacher on-policy distillation: each prompt is routed to the expert for its domain, and that expert supplies a dense per-token log-probability reward on the student’s own samples, extending single-teacher on-policy distillation (OPD) (Li et al., 2026) to several experts at once. Ma et al. (2026a) formalise it as a post-training stage for exactly this consolidation, and industrial post-training reports build on the same combination of multi-teacher distillation and RL (Shao et al., 2024; Yu et al., 2025; Bercovich et al., 2025). However, what the community does not have is an open recipe for that stage, and the mechanism behind it remains unexplored.

The Open-MOPD recipe. We therefore build one from scratch: a three-domain mixed SFT on SmoLM3-3B-Base, math / code / instruction-following (IF) RL teachers forked from that same start, and multi-teacher OPD that routes hard on domain labels. The base model is chosen to balance researchability against attributability (Section 2.2). Researchability sets an explicit budget constraint—the end-to-end pipeline and every mechanism ablation must

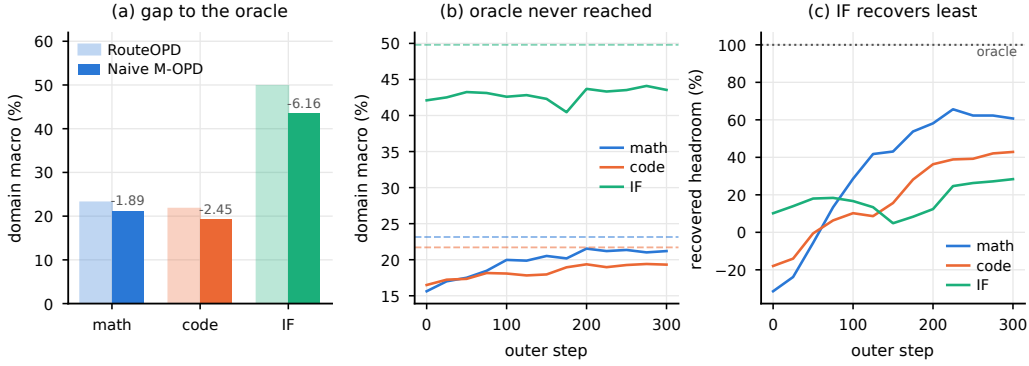


Figure 1: The integration gap is real and strongly asymmetric. (a) Per-domain macro-averages of Naive M-OPD and RouteOPD, the number above each bar giving the gap in that domain; (b) per-domain validation trajectories of Naive M-OPD, the dashed lines being the RouteOPD oracle of the corresponding domain; (c) recovered headroom relative to the available headroom, $(\text{current} - \pi_{\text{mixsft}})/(\text{oracle} - \pi_{\text{mixsft}})$. IF has the largest absolute gap and the lowest recovered headroom, and is the first to stop improving.

run repeatedly within a single $8 \times \text{A100-80GB}$ node—since multi-teacher OPD needs student generation, teacher forwards, long-response training and per-domain validation at once. Attributability sets a floor on capacity: a smaller model truncates long-chain supervision before it reaches a usable trajectory, which would confound optimisation imbalance with a student that simply cannot finish an answer.

Phenomenon and attribution. Under a single six-benchmark protocol, naive multi-teacher OPD reaches 28.05 while distilling each domain on its own reaches 31.55, an oracle that routes by domain and therefore cannot be deployed as one model; we call the 3.50-point loss the capability integration gap, and it is strongly asymmetric (Figure 1): IF falls 6.16 points below its own oracle, 3.3 times the math gap, and is the domain that stops improving earliest. Per-domain measurement locates the gap in how optimisation budget is allocated, along three independent axes. Prompt share is not update share: the loss aggregates over tokens, and IF answers are far shorter than math and code ones, so IF holds 20.3% of the prompts while contributing 0.99% of the gradient tokens. Equalising the token budget does not hold it, because the reward a teacher can still supply shrinks as its student closes in, and it shrinks at a different rate in each domain, so the budget drifts apart again within tens of steps. The third axis comes from a throughput optimisation: reusing one batch of samples for several gradient updates leaves the reward measured against the student that produced the samples rather than the student being updated.

Fix. Open-MOPD repairs one axis per mechanism, so each can be validated on its own. Token-share balancing makes the token budget of a domain a quantity we set rather than a by-product of how long its answers happen to be. Where that budget should go we first answered in the conventional way, by normalising the reward scale across domains. That balances the wrong quantity: it feeds the domains that have already caught up, whose gap then shrinks further and draws in more budget still, until the run collapses at step 74. Gap-following allocation runs the other way, spending the budget where the student has the most left to learn from its teacher; the direction is not a knob to tune but the difference between convergence and collapse. Reward refresh recomputes the student-dependent part of the reward before each update while reusing the cached teacher part, following how AsyncOPD (Kang et al., 2026) handles staleness. Together the three lift a single deployable student from recovering 35.6% of the headroom that domain-routed experts enjoy to recovering 83.4% of it.

Contributions.

- We give a quantitative characterisation of the capability integration gap in multi-teacher OPD, and attribute it with measurable evidence to the allocation of optimisation budget (Section 3).
- We decompose that imbalance into three actionable axes—the cross-domain token budget, the gap that moves as distillation proceeds, and reward staleness inside a reused batch—and give an update rule for each (Section 4).
- We validate the three mechanisms one at a time on a single cumulative ladder: each mechanism is first confirmed to change the quantity it claims to control, then credited with its score gain in Table 4; the allocation direction of gap-following allocation is shown to be structural (Section 5).
- We release the complete recipe—every hyperparameter from base-model choice through mixed SFT and domain RL teachers to multi-teacher OPD (Appendix A)—together with the raw training data and plotting scripts behind every figure, so that these conclusions can be checked directly without first rebuilding the whole upstream pipeline.

2 OPEN-MOPD: AN OPEN RECIPE FOR MULTI-TEACHER CAPABILITY INTEGRATION

Our goal is to integrate the capabilities of several domain experts into one independently deployable student model. Let the set of domains be $\mathcal{D}$. On our base model, $\mathcal{D} = \{\text{math}, \text{code}, \text{instruction_following (IF)}\}$. Each domain has an expert teacher π_{ϕ_d} trained with reinforcement learning, and the student model is π_θ . Training samples carry their own domain label $d(x)$, so the router does not have to be learnt: math problems go to the math teacher, code problems to the code teacher, and instruction-following problems to the IF teacher. With routing given, the question Open-MOPD asks is whether the capabilities of the three experts can be written stably into the same student.

2.1 THE MULTI-TEACHER ON-POLICY DISTILLATION OBJECTIVE

For a prompt x , the student first samples a response $y \sim \pi_\theta(\cdot | x)$ from the current policy. The teacher does not generate new responses; it only supplies a distribution on the states the student actually visits. Write the routed teacher as $\pi_{\phi_{d(x)}}$. At position t , we take the top- k token set of the student distribution

$$\mathcal{S}_t = \text{TopK}_k(\pi_\theta(\cdot | x, y_{<t})), \quad (1)$$

with $k = 16$ by default. For each $v \in \mathcal{S}_t$, we first define the teacher-student log-probability difference

$$\delta_t(v) = \text{sg}[\log \pi_{\phi_{d(x)}}(v | x, y_{<t}) - \log \pi_\theta(v | x, y_{<t})], \quad (2)$$

where $\text{sg}[\cdot]$ denotes stop-gradient; this is the token-level advantage signal of dense distillation. It is then aggregated with softmax weights over $\mathcal{S}_t$ into a dense reward

$$r_t(v) = \delta_t(v) \cdot \tilde{\pi}_\theta(v | x, y_{<t}), \quad \tilde{\pi}_\theta(v | x, y_{<t}) = \text{softmax}_{u \in \mathcal{S}_t}[\log \pi_\theta(u | x, y_{<t})](v), \quad (3)$$

and tokens outside $\mathcal{S}_t$ contribute zero. The position-level reward is

$$r_t = \sum_{v \in \mathcal{S}_t} r_t(v). \quad (4)$$

With no critic, r_t is placed directly in the advantage slot of PPO. Intuitively, $\delta_t(v) > 0$ means that the teacher endorses that token more than the student does; $\tilde{\pi}_\theta$ only determines the share each token inside the top- k contributes to the position-level reward r_t . Because the response is always generated by the student, the supervision distribution moves together with the student; this is precisely the fundamental difference between on-policy distillation and supervised fine-tuning on offline teacher trajectories.

Section 3 starts from this minimal version, which carries no additional balancing mechanism, so that later changes can be traced back to the same baseline.

2.2 WHY SMOLLM3-3B

The open recipe uses SmolLM3-3B-Base as its base model. It is a fully open 3B decoder-only model whose pre-training covers web, math and code data and which is trained at a 64K context (Hugging Face, 2025). This choice balances *researchability* against *attributability*.

Researchability. Multi-teacher OPD needs student generation, one or more teacher forwards, long-response training and per-domain validation at once; a full reproduction also includes mixed-domain SFT, three domain RL teachers and the final multi-teacher distillation. If the base model started at 7B or larger, that chain would be hard to close within a budget on which an ordinary academic team can ablate repeatedly: the GPU hours taken by a single experiment rise, and the repeated runs needed for mechanism comparisons become unaffordable. Open-MOPD therefore takes “runnable on a single 8×A100-80GB node” as a recipe constraint, so that the end-to-end pipeline and the mechanism ablations can be executed repeatedly within the same resource boundary.

Attributability. Attributability is first of all a capacity question. A model that is too small lets the response length limit truncate long-chain supervision before it enters a usable trajectory; the failures observed then cannot be separated into optimisation imbalance and “the student simply cannot fit a complete answer”. We observed this failure mode earlier with the smaller Qwen3-1.7B-Base as base model: across five epochs of SFT on the same OpenR1-Math-93k, its best AIME24 result was only 7.08%, and even with a 31K generation budget the truncation rate stayed at 69.17–80.42%. Qwen2.5-7B-Base on the same data reached 31.25%, with the truncation rate down to 12.92% (Appendix A.1). On too small a model, therefore, a high truncation rate deprives later mechanism conclusions of a unique explanation. SmolLM3-3B-Base raises capacity to the point where it carries the long-chain supervision required by the 32,768-token math / code SFT limit used here, providing observation conditions for the integration gap and the mechanism ablations below that are not dominated by length truncation.

2.3 THE END-TO-END TRAINING RECIPE

The Open-MOPD recipe is built on SmolLM3-3B-Base in three consecutive stages; the teachers and the student of the latter two stages all fork from the same mixed-domain SFT start, so that cross-domain differences can be traced to domain RL rather than to different base models or tokenizers. Table 1 summarises the base model, the data and the evaluation entry points of the recipe.

Stage I: mixed-domain SFT. Starting from SmolLM3-3B-Base, we balance the sample count of each domain by response token count and then run four epochs of supervised fine-tuning on the mixed data, giving the shared student checkpoint π_{mixsft} . The data mixture is: math OpenR1-Math-93k (93,733 examples), code OCR-50k sampled from the full OpenCodeReasoning set (50,000 examples), and instruction-following Instruction-Nemotron aligned (820,039 examples); estimated by response tokens, the shares are approximately 37% math, 28% code and 35% IF. The sequence length limit is 32,768.

Stage II: per-domain RL teachers. Three teachers are initialised separately from π_{mixsft} , each running RL only on the verifiable reward of its own domain. The math teacher is trained on DAPO-Math-17k; the code teacher on DeepScaler-24k (the LiveCodeBench-decontaminated version); the IF teacher on Nemotron-IF-RL-46k. This stage does not mix domains: each teacher changes only the capability of its own domain, giving $\pi_{\phi_{\text{math}}}, \pi_{\phi_{\text{code}}}, \pi_{\phi_{\text{IF}}}$.

Stage III: multi-teacher on-policy distillation. The student is again initialised from π_{mixsft} . Training prompts come from the union of the three domains and are sampled as a domain mixture; each prompt is routed to the teacher of its domain and distilled on-policy under the objective of Section 2.1, with the three domains jointly updating the same set of student parameters π_{θ} . The response length limit is 16K for math and code and 2K for IF; by default the dense reward is computed on the student top- k ($k = 16$).

Table 1: The Open-MOPD recipe. The three stages are chained in order and the product of each stage is the input of the next: mixed-domain SFT gives the shared start π_{mixsft} , domain RL forks three teachers π_{ϕ_d} from that start, and multi-teacher OPD then trains the student π_θ from the same start. All datasets are publicly available (links point to the download locations), so the entire recipe can be reproduced in full.

Stage	Data	Key settings and products
Base model	https://huggingface.co/HuggingFaceTB/SmolLM3 3B-BaseSmolLM3-3B-Base	Shared jointly by the trajectory-closure failure at 1.7B and the ablation cost at 7B (Appendix A.1)
I Mixed-domain SFT	Math: https://huggingface.co/datasets/open-r1/OpenR1-Math-220k OpenR1-Math-93k Code: https://huggingface.co/datasets/nvidia/OpenCodeReasoningOCR-50k (sampled from the full set) IF: https://huggingface.co/datasets/nvidia/Llama-Nemotron-Post-Training-DatasetInstruction-Nemotron	Four epochs; sequence limit 32,768; approximately balanced by response tokens (37/28/35) product π_{mixsft}
II Domain RL teachers	Math: https://huggingface.co/datasets/BytedanceSIA/DAPO-Math-17k Code: https://huggingface.co/datasets/agentica-org/DeepCoder-Preview-DatasetDeepScaler-24k (LCB-decontaminated) IF: https://huggingface.co/datasets/nvidia/Nemotron-Cascade-2-RL-dataNemotron-IF-RL-46k	Independent RL per domain, no π_{mixsft} mixing; all forked from π_{mixsft} products $\pi_{\phi_{\text{math}}}, \pi_{\phi_{\text{code}}}, \pi_{\phi_{\text{IF}}}$
III Multi-teacher OPD	The union of the three domains' RL prompts above; each prompt is hard-routed to its teacher by domain label	Response limit 16K for math/code and 2K for IF; top- $k=16$; the student is initialised from π_{mixsft} product π_θ
Evaluation	Math: https://huggingface.co/datasets/HuggingFaceH4/llm-as-a-judge Code : https://huggingface.co/datasets/livecodebench/evaluation IF : https://huggingface.co/datasets/google/IFEval https://huggingface.co/datasets/allenai/IFBench_test	mean@64 / mean@10 / mean@1; Each domain only 0.4 of SFT data datasets within a domain first, then all domains together three domains https://huggingface.co/datasets/livecodebench/evaluation https://huggingface.co/datasets/google/IFEval https://huggingface.co/datasets/allenai/IFBench_test

2.4 REFERENCE FRAME AND UNIFIED EVALUATION PROTOCOL

Capability integration needs a reference frame different from an ordinary model leaderboard. We use *RouteOPD* to denote a non-deployable oracle: a single-teacher OPD student is trained for each domain, and at evaluation time the true domain label selects the corresponding student. RouteOPD does not yield one unified model, but it answers the key question: without requiring parameter sharing, how much capability this set of teachers and this OPD objective have, at most, already achieved. The gap between a single M-OPD student and RouteOPD therefore defines the *integration gap* of this paper.

Table 2: The capability integration gap on SmolLM3-3B. Each domain reports its sub-datasets and its within-domain macro-average; total score = the simple average of the three domain macro-averages. The four SFT rows are the per-epoch checkpoints; single-domain RL / OPD fill in their own domain only; the grey RouteRL / RouteOPD are oracles that route hard by domain and are not deployable. Recovery rate = $\frac{\text{row total} - \text{SFT}}{\text{RouteRL} - \text{SFT}}$, with the SFT baseline taken as π_{mixsft} (e4) in the table. Within the Baselines and M-OPD blocks, the highest score in each column is in bold and the second highest is underlined.

Method	Math (avg@64)			Code (avg@10)			IF			Total	Recovery rate
	AIME24	AIME25	avg	LCBv5	LCBv6	avg	IFEval	IFB _{test}	avg		
SmolLM3-3B-Base	2.08	1.72	1.90	3.41	6.17	4.79	16.08	13.00	14.54	7.08	—
<i>SFT</i>											
π_{mixsft} (e1)	12.24	16.61	14.43	13.29	17.20	15.25	64.70	16.00	40.35	23.34	—
π_{mixsft} (e2)	12.55	17.97	15.26	13.17	16.63	14.90	65.62	16.00	40.81	23.66	—
π_{mixsft} (e3)	15.10	19.53	17.32	14.97	18.57	16.77	66.36	18.00	42.18	25.42	—
π_{mixsft} (e4)	15.63	20.26	17.95	15.99	19.20	17.60	66.91	16.00	41.46	25.67	—
<i>RL</i>											
$\pi_{\phi_{\text{math}}}$	23.65	24.84	24.24	—	—	—	—	—	—	—	—
$\pi_{\phi_{\text{code}}}$	—	—	—	22.16	21.31	21.73	—	—	—	—	—
$\pi_{\phi_{\text{IF}}}$	—	—	—	—	—	—	74.49	27.67	51.08	—	—
RouteRL	23.65	24.84	24.24	22.16	21.31	21.73	74.49	27.67	51.08	32.35	100%
<i>OPD</i>											
Math-OPD	22.34	23.96	23.15	—	—	—	—	—	—	—	—
Code-OPD	—	—	—	22.28	21.14	21.71	—	—	—	—	—
IF-OPD	—	—	—	—	—	—	75.60	24.00	49.80	—	—
RouteOPD	22.34	23.96	23.15	22.28	21.14	21.71	75.60	24.00	49.80	31.55	88.0%
<i>Baselines</i>											
RFT	22.97	23.91	23.44	18.98	19.43	19.21	55.08	18.67	36.87	26.51	12.6%
π_{mixrl}	21.15	22.14	21.64	16.59	20.97	18.78	70.24	<u>22.67</u>	46.45	28.96	49.3%
ParamMerge-Avg	18.91	20.99	19.95	18.38	21.20	19.79	70.06	17.67	43.86	27.87	32.9%
ParamMerge-TA	21.93	22.76	22.34	21.74	23.14	22.44	<u>71.53</u>	21.67	<u>46.60</u>	<u>30.46</u>	<u>71.7%</u>
<i>M-OPD</i>											
Naive M-OPD	20.92	21.60	21.26	17.53	20.99	19.26	68.61	18.67	43.64	28.05	35.6%
Open-MOPD (Ours)	<u>21.98</u>	<u>22.86</u>	<u>22.42</u>	<u>20.84</u>	<u>22.63</u>	<u>21.73</u>	74.49	24.67	49.58	31.24	83.4%

All main results use the same online verifier and the same six benchmarks, so that differences in the evaluation harness are not mistaken for algorithmic gains. Scoring is done per dataset first, then averaged simply over the datasets within a domain, and the final total score is the simple average of the three domain macro-averages. The math domain uses AIME24 and AIME25 with $n = 64$ and temperature = 0.6, reporting accuracy mean@64; the instruction-following domain uses IFEval and IFBench_{test} with $n = 1$, reporting accuracy mean@1; the code domain uses LiveCodeBench v5 and v6 with $n = 10$ and temperature = 1.0, reporting accuracy mean@10.

Table 2 summarises the SmolLM3-3B reference ladder under this protocol: the base model and each epoch of mixed-domain SFT; single-domain RL / OPD and their hard-routed combinations RouteRL / RouteOPD (grey, not deployable as one model); the RFT, π_{mixrl} and Parameter Merging comparisons; and Naive M-OPD together with our Open-MOPD. The diagnosis and method comparisons that follow all take this table as their reference.

3 CAPABILITY INTEGRATION IS NOT A SIMPLE SUM OF EXPERT CAPABILITIES

The recipe of Section 2 looks as though it should work smoothly: the teachers are all trained from the same student starting point, the domain labels give error-free routing, and the student receives dense supervision on its own distribution. If every prompt receives the signal of its own domain’s teacher only, then mixing three single-teacher OPD tasks together appears to be no more than an engineering extension. Our experiments show that this intuition is wrong. The main difficulty of multi-teacher OPD lies not in whether routing is correct, but in how several domains compete for the same optimisation budget.

3.1 A SIMPLE BASELINE EXPOSES THE INTEGRATION GAP

In Table 2, Naive M-OPD lifts π_{mixsft} from 25.67 to 28.05, which shows that the multi-teacher signal is not ineffective; but it still falls 0.91 points below π_{mixrl} and, more importantly, 3.50 points below RouteOPD. RouteOPD already reaches 31.55, so the teachers, the student initialisation and the single-teacher distillation objective together supply enough capability; the capability is lost when it is written into one and the same student.

Figure 1 decomposes this gap across domains. The gap is not evenly distributed: IF falls 6.16 points below its own single-teacher oracle, 3.3 times the math gap (1.89), and is also the main source of the total integration gap (Figure 1a). Along the training trajectory (Figure 1b), none of the three domains touches its own oracle within 300 steps. Once the scores are converted into the “fraction of the headroom available to that domain that has been recovered” (Figure 1c), over the [200, 300] window math recovers 64% of the headroom up to its own oracle, code 40%, and IF only 26%; IF is also the only domain that regresses in the middle of training (only 11% over the [100, 200] window).

3.2 A FIRST HYPOTHESIS: TEACHER CONFLICT

A natural hypothesis is that teachers of different domains give mutually contradictory token preferences on the same student trajectory: although the math, code and IF teachers are each selected on their own domain only, they share a large number of formatting words, connectives and reasoning templates; if the teachers disagree on these generic tokens, hard routing may push the same kind of state in different directions and thereby produce interference in the shared parameters.

Multi-teacher on-policy distillation makes this hypothesis directly measurable. For every token sampled by the student, we let all three teachers score the same context, and define

$$c_t = \max_{d \in \mathcal{D}} \log \pi_{\phi_d}(y_t \mid x, y_{<t}) - \min_{d \in \mathcal{D}} \log \pi_{\phi_d}(y_t \mid x, y_{<t}) \quad (5)$$

as the teacher disagreement. If conflict were a systematic bottleneck, c_t would be large on a substantial fraction of tokens, and masking high-conflict tokens or switching to a teacher consensus should improve training.

Figure 2 gives three mutually independent pieces of evidence, which consistently reject this hypothesis. First, the magnitude of the disagreement is small and stable: the average c_t over the whole run is 0.126 nat and stays below 0.27 nat throughout the 300 training steps, more than an order of magnitude below a conflict criterion of 1 nat (at which the probability ratio between the highest and the lowest teacher on that token is about $e \approx 2.7$) (Figure 2a). Second, high-conflict tokens form a thin tail: on average only 0.62% of tokens satisfy $c_t > 1$; even on IF, where disagreement is largest, the figure is 3.9%, and on math it is as low as 0.31% (Figure 2b). A maximum of 30.58 nat shows that extreme conflict does exist, but it is concentrated on very few tokens rather than covering the bulk of training. Third, direct intervention brings no benefit. We implement two kinds of mechanism: the first is a conflict mask—by c_t quantile, the top 1%, 5% and 20% of tokens are removed from the distillation loss (with thresholds 0.83, 0.49 and 0.22 nat respectively), and the weights of the remaining tokens are rescaled to preserve the total loss scale, which rules out the possibility that the threshold was too high and the mechanism never fired; the second is consensus—agreement and conflict are separated at a fixed threshold of 1 nat, on tokens with $c_t \leq 1$ the hard-routed target is replaced by the average of the three teachers’ log-probs, and on $c_t > 1$ the routed teacher is retained (purple points in Figure 2c). Compared pairwise at exactly the same validation steps as the baseline, the total-score changes of the three masks are -0.52 , -0.73 and -0.75 ; that of consensus is -0.83 . The conflict signal is measured accurately and the interventions do take effect, yet their direction is uniformly negative.

This negative result pushes the question to a finer granularity. Hard routing has already settled which teacher each prompt should listen to; what remains is how many effective updates each domain receives in total—that is, how the optimisation budget is allocated across domains.

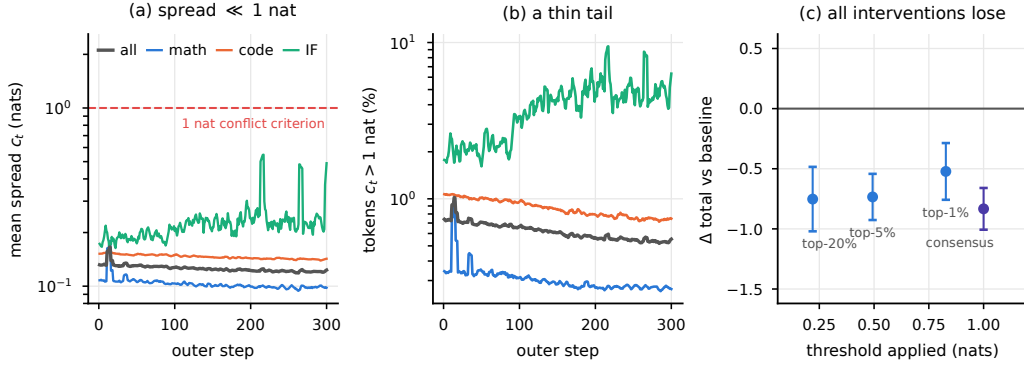


Figure 2: Teacher conflict can be measured, but it is not the bottleneck. (a) Average teacher disagreement c_t (log axis) and its distance to the 1 nat criterion; the legend applies to both panels (a) and (b); (b) fraction of tokens satisfying $c_t > 1$, decomposed by domain; (c) total-score change of four conflict interventions relative to the same baseline. Blue points are the conflict mask, with the horizontal axis giving the threshold when the top $k\%$ of tokens by c_t quantile are removed (the larger k , the lower the threshold); purple points are consensus, with the threshold fixed at 1 nat—on tokens with $c_t \leq 1$ the hard-routed target is replaced by the average of the three teachers’ log-probs, and on $c_t > 1$ the routed teacher is retained. Error bars are validation standard error.

3.3 PER-DOMAIN MEASUREMENT REVEALS THE REAL IMBALANCE

To locate where the budget goes, we record within a single training run the number of prompts, the number of valid response tokens, the average response length and the per-token reward magnitude of every domain. The three sets of measurements expose three independent axes of imbalance.

Prompt share is not update share. The training loss uses token-mean aggregation, so every valid response token contributes to the loss once. The raw token share of domain d is therefore

$$s_d^{\text{tok}} = \frac{n_d L_d}{\sum_{j \in \mathcal{D}} n_j L_j}, \quad (6)$$

where n_d is the number of prompts of that domain and L_d is the average response length. Figure 3 shows that the two shares are completely decoupled along the whole trajectory: the prompt share is fixed by the sampler at 39.8%/39.8%/20.3%, whereas the token share is 49.7%/49.3%/0.99% (Figure 3a). Across all 300 steps the token share of IF never exceeds 1.65% and never falls below 0.44%, so this is not an incidental property of a particular batch. The cause is given directly by length (Figure 3b): the average response length of math and code is about 10,500 tokens while that of IF is only 409 tokens, a difference of more than 25 times; Equation equation 6 is therefore almost entirely determined by length. This also shows that simply raising the sampling frequency of IF is not a viable fix: for IF to obtain 1/3 of the token budget, its number of prompts would have to be scaled up about 33.6 times (Figure 3c), and the math and code prompts in a batch would be squeezed to the point where long-chain supervision can no longer be maintained.

The update budget also changes with distillation progress. The token count only determines how many times a domain appears; the strength of a single update also depends on the reward magnitude. As a first-order proxy for the budget, the contribution of a domain satisfies

$$B_d \propto s_d^{\text{tok}} \bar{m}_d, \quad \bar{m}_d = \mathbb{E}_{t \in d}[|r_t|]. \quad (7)$$

Here $\bar{m}_d$ is the average per-token reward-signal magnitude on domain d : it is the expectation of $|r_t|$ over the tokens of that domain and characterises the average strength that one appearance contributes to the parameter update. Figure 4 establishes two facts. First, $\bar{m}_d$

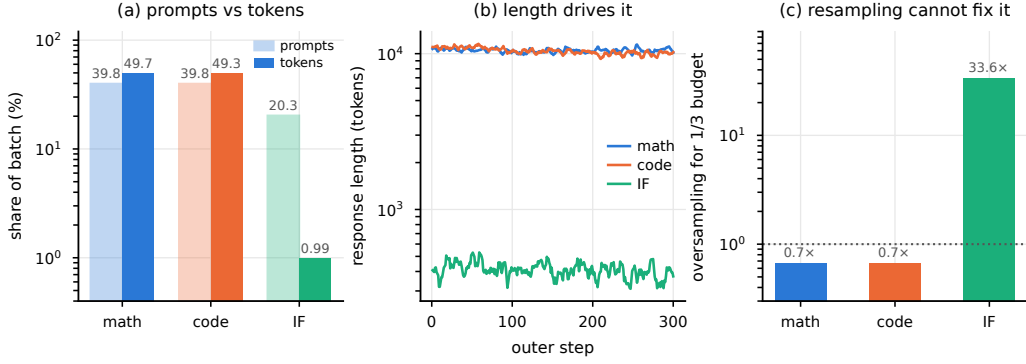


Figure 3: Prompt share and gradient-token share are completely decoupled. (a) Whole-run averages of the prompt share and the token share of each domain (log axis; light bars are prompts, solid bars are tokens): IF holds about 20% of the prompts yet contributes only about 1% of the gradient tokens; (b) the average response length (log axis) explains the entire difference, about 10,500 tokens for math/code against about 409 tokens for IF; (c) if oversampling alone were used to bring every domain to 1/3 of the token budget, IF would need a $33.6\times$ prompt amplification, and the math and code prompts in a batch would be squeezed to the point where long-chain supervision can no longer be maintained.

differs by 4.9 times across domains: early in training it is 0.019 for math, 0.063 for code and 0.091 for IF (Figure 4a), so even when the token share is flattened the per-token update strength remains unequal. Second, and more importantly, $\bar{m}_d$ is not a domain constant. The dense reward in Equation equation 3 contains the teacher–student gap directly; as a domain approaches its teacher this gap naturally shrinks, and it shrinks at a different rate in each domain: along the same trajectory IF shrinks by 2.4 times, math by 2.1 times and code by 1.9 times (Figure 4b). The consequence of combining the two is clearest once the first axis of imbalance has been fixed. When s_d^{tok} is forced flat to 1/3, only the $\bar{m}_d$ term of Equation equation 7 remains, and the residual variation in share is caused entirely by the gap: within 25 steps the budget share of IF drops from 48.7% to around 9% and ends at 11.4%, while that of code rises from 39.6% to 63.8% (Figure 4c). In other words, a flattened token budget is pulled apart again by the gap within tens of steps, and in the direction opposite to the initial configuration. Statically tuning the domains to some initial ratio only holds at the first step.

Throughput optimisation introduces a third axis of imbalance. To amortise the cost of generation, practical training usually performs K inner updates from one large rollout batch. The teacher is fixed here, but the student has already changed after the first inner update; if the subsequent updates still use the rollout-time $\log \pi_\theta(v \mid x, y_{<t})$ and $\tilde{\pi}_\theta(v \mid x, y_{<t})$ ($v \in \mathcal{S}_t$), then the student-dependent part of Equation equation 3 is inconsistent with the current policy. This shift is measurable: Figure 5 measures it by the KL between the rollout policy and the current policy within the same batch, and its value rises monotonically with K , growing from 0 at $K=1$ (no shift by definition) to 0.059 at $K=4$ and 0.216 at $K=32$ (Figure 5a); the fraction of tokens clipped by PPO rises in step from 0 to 0.86 (Figure 5b). In other words, under a high-throughput configuration most tokens are updated on a student that has already drifted away from the rollout policy, while the dense reward is still defined by the student at rollout time. This problem does not exist at $K=1$, but it is systematically amplified by throughput optimisation.

The three sets of measurements decompose a vague question of “multiple teachers interfering with each other” into three actionable objects: the cross-domain token budget, the teacher–student gap that changes with learning progress, and the reward staleness inside inner updates. The next section gives the corresponding fix for each.

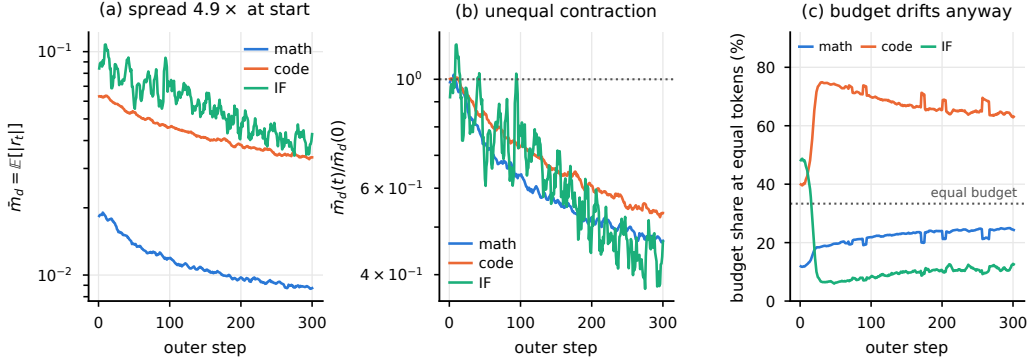


Figure 4: The effective budget drifts with distillation progress. (a) Per-token reward magnitude $\bar{m}_d$ of each domain (log axis), differing by $4.9\times$ early on; (b) after normalising to their respective initial values, the three domains shrink at different rates, which shows that $\bar{m}_d$ measures the remaining teacher–student gap; (c) on a trajectory where s_d^{tok} is flattened to $1/3$, Equation 7 degenerates to $B_d \propto \bar{m}_d$, so the curves show the budget drift caused by the gap alone: within 25 steps the share of IF falls below the dashed line to about 9%, while code rises to 63.8%. Flattening the token share does not lock the budget in place.

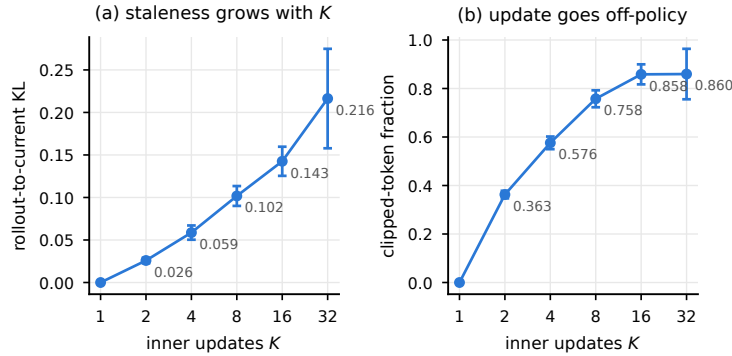


Figure 5: Throughput optimisation turns reward staleness into a third axis of imbalance. The horizontal axis is the number of inner updates K per rollout batch. (a) Within the same batch, the KL between the rollout policy and the current student policy grows monotonically with K ; (b) the fraction of tokens clipped by PPO rises in step. At $K=1$ both are 0, and the dense reward is then consistent with the current student.

4 FROM MEASUREMENT TO FIXES: OPEN-MOPD

Open-MOPD keeps the hard routing and the on-policy distillation objective of Section 2.1, and changes only how the optimisation budget is allocated, updated and refreshed. The three mechanisms correspond to the three measurements of Section 3.3: *token-share balancing* controls the domain token budget within a batch, *gap-following allocation* adjusts the budget over the course of training according to the remaining distillation gap, *reward refresh* refreshes the student-dependent reward across the several inner updates of one rollout. They act on different time scales, so they can be validated independently and can also be combined into a single training recipe. Figure 6 gives the complete method: the data flow of one training iteration, with badges marking the stage that each mechanism rewrites.

Overview of OpenMOPD

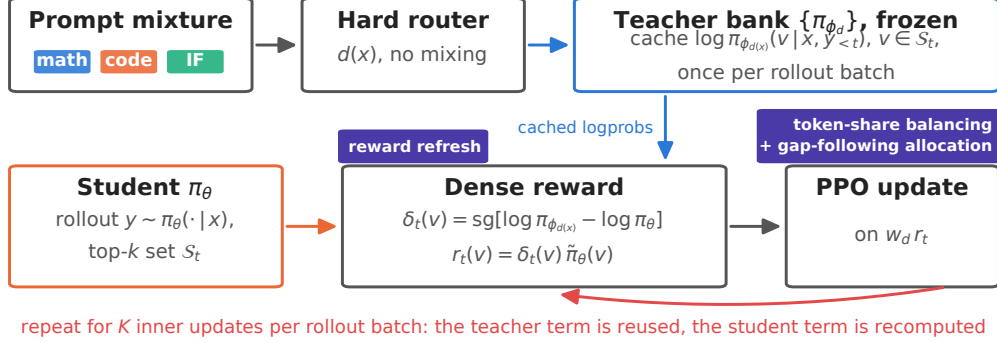


Figure 6: Overview of Open-MOPD. Prompts are hard-routed by domain label, the teacher computes log-probabilities on $\mathcal{S}_t$ once at rollout time and caches them, the student generates answers on its own distribution, the two together form the dense reward, which then enters the PPO update; within one rollout batch the inner update is repeated K times, with the teacher term reused and the student term recomputed by reward refresh. The badges mark the stage each mechanism rewrites: token-share balancing and gap-following allocation determine the domain loss weight w_d , and reward refresh rebuilds r_t within every inner update.

4.1 TOKEN-SHARE BALANCING

Token-share balancing gives every domain a pre-specified share of the token-mean loss. Let the target domain budget be g_d^* , with $\sum_d g_d^* = 1$. In every batch we compute s_d^{tok} from the true attention mask, and then apply to the token loss of domain d

$$w_d^{\text{share}} = \frac{g_d^*}{s_d^{\text{tok}}}. \quad (8)$$

After weighting, the effective share of domain d is $w_d^{\text{share}} s_d^{\text{tok}} / \sum_j w_j^{\text{share}} s_j^{\text{tok}} = g_d^*$. The mechanism therefore controls directly the quantity the loss actually sees, without relying on the response length being stable and without needing extreme oversampling to compensate for short answers.

The main recipe uses the equal-share target $g^* = (1/3, 1/3, 1/3)$, which needs no corpus-specific tuning. On the token shares shown in Figure 3, it yields weights of 0.69 for math, 0.66 for code and 32.7 for IF: every IF token is amplified about 48 times to compensate for its 25-fold length disadvantage, while Equation equation 8 guarantees that the weighted shares of the three domains are exactly 33.33%. This check is completed before any capability score is read: whether the mechanism takes effect is verified by the realised shares, not inferred backwards from the final score.

Token-share balancing corrects only “how many times a domain appears” and does not attempt to remove differences in the strength of individual updates. This boundary is deliberate: the reward magnitude contains the teacher–student gap, and crudely normalising it to a common scale would delete information about learning progress.

4.2 GAP-FOLLOWING ALLOCATION

Let m_d be the running per-token reward-magnitude statistic of domain d , for example an exponential moving average of $|r|$; it is an observable proxy for the remaining teacher–student gap. One immediate idea is to normalise with $m_d^{-\alpha}$, so that domains with a smaller initial reward receive a larger weight. This direction looks reasonable at the start of training: the m_d of math is only about one third of that of code. However, m_d also measures the

remaining distillation gap; the closer a domain is to its teacher, the smaller its m_d , so inverse normalisation in fact gives more budget to domains that have already converged.

This constitutes a self-reinforcing loop: the closer a domain is to its teacher, the smaller its m_d , the larger its $m_d^{-\alpha}$, and the more budget it receives, so it approaches its teacher faster still and its m_d shrinks further. The loop has no restoring term, so the weights diverge monotonically until training destabilises. It appears quickly on a real trajectory: the reward magnitude of IF shrinks by 35.3 times over the first 75 steps; at $\alpha = 0.5$, the IF weight of the inverse version grows from 26.7 to 90.7, while the code weight falls from 0.44 to 0.27. This failure is not a hyperparameter issue but a wrong update direction: the denominator shrinks automatically with convergence, so any positive α preserves the same loop.

Gap-following allocation keeps the direction of the gap, interprets it as “capability not yet distilled”, and allocates the budget to the domains that still have a larger gap:

$$\tilde{w}_d = w_d^{\text{share}} \cdot \text{Clamp}\left(\left(\frac{m_d}{m_{\text{ref}}}\right)^\alpha, 0.05, 20\right), \quad w_d^{\text{gap}} = \frac{\tilde{w}_d}{\sum_j \tilde{w}_j s_j^{\text{tok}}}, \quad (9)$$

where m_{ref} is the cross-domain mean of the current batch and α controls the strength of the dynamic allocation. Clamp acts on the gap factor only, confining it to a band of $400\times$, so that a collapse of the reward magnitude in a single domain cannot drain the entire budget; the share correction set by Equation equation 8 is itself not clipped. The divisor $\sum_j \tilde{w}_j s_j^{\text{tok}}$ is the token-share-weighted mean of the weights, so after normalisation $\sum_d w_d^{\text{gap}} s_d^{\text{tok}} = 1$: the rule changes only the relative shares between domains and does not change the total loss scale of a batch, and therefore is not equivalent to silently adjusting the learning rate. Both the shares and the normalisation are computed only over the domains present in the current batch—an absent domain cannot be allocated any gradient—so the correction does not rely on cross-batch length estimates. When a domain approaches its teacher, its m_d and its budget fall together; domains that still have a larger teacher–student gap receive more updates. It is therefore an allocation rule that follows the remaining gap, not reward normalisation: it redirects the budget fixed by token-share balancing towards the domain with the largest distillation headroom. Section 5.2 gives its benefit when added on top of token-share balancing alone, together with the collapse that occurs when α takes a negative sign.

4.3 REWARD REFRESH

The first two mechanisms deal with the budget between domains; reward refresh deals with temporal inconsistency inside one rollout. The idea of this mechanism is borrowed from AsyncOPD (Kang et al., 2026): in asynchronous OPD the cached teacher term can be reused, while the student-dependent term should be recomputed at learner time with the current student. We apply the same treatment to the K inner updates of a synchronous pipeline—there the staleness does not come from asynchronous rollout, but from the student being updated K times in a row within one rollout batch. Let one rollout batch be split into K minibatches. The teacher parameters are fixed, so $\log \pi_{\phi_{d(x)}}(v \mid x, y_{<t})$ for the tokens in the rollout-time set $\mathcal{S}_t$ needs to be computed and cached only once; the student parameters, by contrast, change after every inner update. In the actor forward of the k -th inner update, reward refresh refreshes the student-dependent term on the same set $\mathcal{S}_t$:

$$\begin{aligned} \delta_t^{(k)}(v) &= \text{sg}\left[\log \pi_{\phi_{d(x)}}(v \mid x, y_{<t}) - \log \pi_{\theta}^{(k)}(v \mid x, y_{<t})\right], \\ r_t^{(k)}(v) &= \delta_t^{(k)}(v) \cdot \tilde{\pi}_{\theta}^{(k)}(v \mid x, y_{<t}), \quad v \in \mathcal{S}_t, \quad k = 0, \dots, K-1. \end{aligned} \quad (10)$$

Here $\tilde{\pi}_{\theta}^{(k)}$ is computed on the current student log-probabilities following Equation equation 3. It is then aggregated into $r_t^{(k)}$ by Equation equation 4 and placed in the advantage slot of PPO. The update is subsequently constructed with the $w_d r_t^{(k)}$ of the current domain, where $w_d = w_d^{\text{gap}}$ when gap-following allocation is enabled and $w_d = w_d^{\text{share}}$ otherwise. The refresh reuses the student forward that PPO has to perform anyway and adds no teacher forward; it only avoids updating a new student with a dense reward defined by an old one.

Table 3: The three mechanisms of Open-MOPD each act on a different axis of imbalance.

Mechanism	Observable	Update rule
token-share balancing	Domain token share of the current batch	Divide the target share by the measured token share, removing the static budget bias caused by response length.
gap-following allocation	Running teacher-student gap of each domain	The budget moves in the same direction as the remaining gap, avoiding the self-reinforcing loop of inverse normalisation.
reward refresh	Current student probabilities at every inner update	Reuse the actor forward to refresh the dense reward, removing the repairable reward staleness inside a minibatch.

At $K = 1$, Equations equation 10–equation 4 coincide with the objective of Section 2.1; at $K > 1$, they repair only the part of the reward that explicitly depends on the student. The sampled student states still come from rollout time, so reward refresh removes the staleness that can be removed without regenerating, rather than turning minibatch training into fully on-policy training.

4.4 THE COMPLETE ALGORITHM

Table 3 summarises what each of the three mechanisms acts on. The key to Open-MOPD is not to ensemble the teacher outputs, but to give the shared student an optimisation budget that is measurable, controllable and updated as training proceeds, while keeping domain hard routing.

One complete iteration performs the following steps in order:

1. sample a batch mixed by domain, and generate on-policy answers with the current student;
2. select the teacher by domain label, and compute and cache $\log \pi_{\phi_d(x)}(v \mid x, y_{<t})$ on the rollout-time $\mathcal{S}_t$;
3. compute s_d^{tok} from the true response mask, and obtain the token-share balancing weight from Equation equation 8;
4. update the gap statistic m_d of every domain, and obtain the dynamic domain weight from Equation equation 9;
5. for every inner update, recompute the current student term, refresh the dense reward by Equation equation 10 and update the student.

This construction forms a closed loop from measurement to intervention: the token share determines token-share balancing, the dynamic gap determines gap-following allocation, and the timeliness of the student-dependent reward determines reward refresh. The experiments of the next section validate three things in turn: whether each mechanism does change the quantity it claims to control; whether that change improves the corresponding domain; and whether the three mechanisms combined narrow the integration gap defined in Section 3.1.

5 ABLATIONS: WHAT EACH MECHANISM DOES

Section 3.3 decomposes the integration gap into three measurable axes of imbalance, and Section 4 gives three corresponding update rules. This section examines them one at a time: whether each mechanism really changes the quantity it claims to control, and whether the three combined narrow the gap defined in Section 3.1. Scores are always taken from the cumulative ladder of Table 4; the non-performance measurements serve only to confirm whether a mechanism has rewritten the budget share, the domain weights or the wallclock composition.

Table 4: The cumulative ladder from Naive M-OPD to the full recipe. Each row changes exactly one thing relative to the row above it, and the change is named in the leftmost column; the three switch columns correspond in order to token-share balancing, gap-following allocation and reward refresh, and K is the off-policy depth.

Configuration	Mechanism			K	Six datasets				
	share	gap	refresh		Math	Code	IF	Total	Δ
Naive M-OPD				1	21.26	19.26	43.64	28.05	—
+share	✓			1	20.55	19.57	47.53	29.22	+1.17
+gap	✓	✓		1	21.00	19.31	49.50	29.94	+1.89
<i>Switching to the $K=4$ throughput setting (256 prompts per update, $4\times$ rollout batch)</i>									
Naive M-OPD (same-setting control)				4	21.62	19.72	46.49	29.28	+1.23
+share+gap	✓	✓		4	23.05	21.07	47.16	30.43	+2.38
Open-MOPD	✓	✓	✓	4	22.42	21.73	49.58	31.24	+3.19

5.1 TOKEN-SHARE BALANCING

The first test has nothing to do with scores: whether the mechanism locks the quantity it claims to lock. With token-share balancing enabled (Section 4.1), the weighted token share $w_d s_d^{\text{tok}}$ stays at 33.33% over all 300 steps (marked by the dashed line in Figure 7a); the same sampler with it switched off hands 99% of the gradient tokens to math and code and leaves only about 1% to IF (Figure 3a). The first axis of imbalance measured in Section 3.3 is therefore removed entirely, and without relying on the response length being stable.

In Table 4 it gives +1.17 points, with the gain coming almost entirely from IF; math and code barely move. Once the budget is flattened, the question that remains is when the budget should go to which domain.

5.2 GAP-FOLLOWING ALLOCATION

Gap-following allocation is added on top of token-share balancing with reward refresh left disabled, so the $K=1$ ladder of Table 4 gives a clean single-mechanism increment of +0.72 points.

The quantity it changes is the trajectory of the share over time. Token-share balancing pins the weighted share at $1/3$, a value insensitive to training progress; gap-following allocation is the only one of the three mechanisms that makes it move with progress (Figure 7a): from the first 25 steps to $[200, 300]$, the math share rises from 15.1% to 32.6%, IF contracts from 34.5% to 17.1%, and code stays above 50% throughout (mean 55.4% over the main segment, peak 87.6%). The ordering is set by $\bar{m}_d$: over the middle window of the token-share-balancing-only trajectory, $\bar{m}_d$ is 0.028 for code, 0.008 for math and 0.003 for IF, that is 2.16, 0.61 and 0.23 times the cross-domain mean; the ratios of the domain weights at the same step are $1.68\times$ for code, $0.90\times$ for math and $0.42\times$ for IF, in the same order. The budget flows to the domain with the largest remaining teacher-student gap.

Once the budget has been flattened, the more natural next move is to flatten the reward scale as well: divide the domain weight by $\bar{m}_d$, so that the effective reward magnitude is equal across domains and no domain takes more gradient merely because its teacher supplies a larger reward. That balances the wrong quantity. Replacing the factor of Eq. equation 9 with the inverse form $\tilde{w}_d = w_d^{\text{share}} \bar{m}_d^{-\alpha}$ ($\alpha=0.5$) sends the budget to the domains whose $\bar{m}_d$ is small, that is, to those already close to their teacher, and the self-reinforcing loop predicted in Section 4.2 appears: the $\bar{m}_d$ of IF contracts by a factor of 32.5 while its domain weight rises from 24.4 to 80.9, and that trajectory terminates at step 74 (Figure 7b). The allocation direction is therefore structural: the sign of α decides whether training is stable, and is not a tunable hyperparameter.

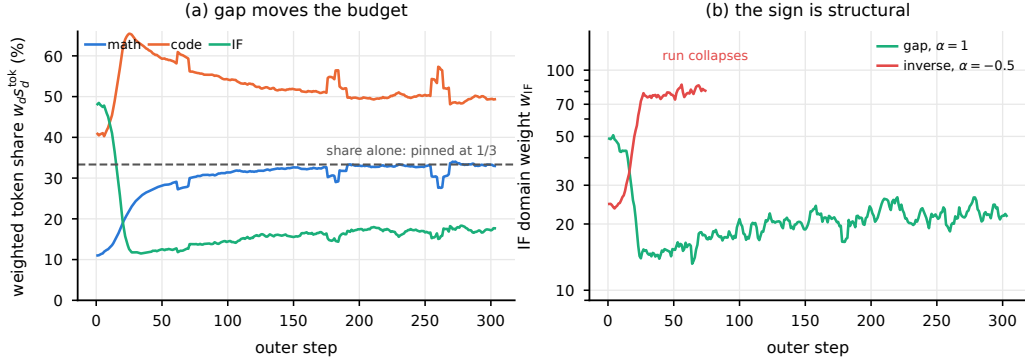


Figure 7: What gap-following allocation rewrites is the budget trajectory, not a hyperparameter knob. (a) Weighted token share $w_{ds_d}^{tok}$: with token-share balancing enabled on its own it is pinned at $1/3$ (dashed line); adding gap-following allocation ($\alpha=1$) makes it move with the remaining teacher–student gap—the domains differ in learning difficulty, so a domain that has already been learnt and has converged loses share, while a domain with more distillation headroom left receives more budget (code rises above 50% here, and IF contracts to about 17%); (b) IF domain weight (log axis): $\alpha=1$ makes it contract, whereas the inverse form ($\alpha=-0.5$) falls into the self-reinforcing loop, and that trajectory terminates at step 74.

5.3 REWARD REFRESH

Reward refresh (Section 4.3) moves the student-dependent part of the dense reward into every inner update. The teacher log-probs are still prefilled once at the top of the rollout batch and reused throughout, and the student log-probs are read off the actor forward that PPO already performs. The change therefore adds no prefill: without refresh the dense reward is prefilled once over the whole rollout batch, with refresh once per inner minibatch, and the K minibatches partition that batch, so under either scheme every token is prefilled exactly once per outer step. What changes is only the student parameters at which that prefill is evaluated.

The measurement agrees. On steady training steps at $K=4$ (steps 25–199, warm-up and validation steps excluded, medians over 168 paired steps), the dense-reward prefill takes 27.3 s with refresh and 27.8 s without, that is 2.10% and 2.12% of one outer step (Figure 8). The step wallclock is 1298 s and 1313 s: a -1.2% difference, negative in sign and smaller than the step-to-step spread, so it is not an overhead attributable to refresh. On top of token-share balancing and gap-following allocation at $K=4$, adding reward refresh gains a further $+0.81$ in Table 4, completing Open-MOPD.

6 RELATED WORK

6.1 ON-POLICY DISTILLATION AND ITS MULTI-TEACHER EXTENSION

Distillation on a fixed corpus. Knowledge distillation trains a student to match a teacher’s output distribution (Hinton et al., 2015); for sequence models the standard instantiation matches the teacher on a corpus of teacher completions (Kim & Rush, 2016). This template compresses encoders (Sanh et al., 2019; Jiao et al., 2019; Wang et al., 2020a), transfers reasoning traces to smaller models (Hsieh et al., 2023; Ho et al., 2022), and has been characterised through capacity gaps (Cho & Hariharan, 2019; Mirzadeh et al., 2019), pre-training design spaces (Peng et al., 2024) and scaling laws (Busbridge et al., 2025), with a documented failure mode when the student is far weaker than the teacher (Li et al., 2025b). Its structural weakness is that the supervision is defined on a distribution the student does not itself produce, the exposure-bias problem of sequence models (Ranzato et al.,

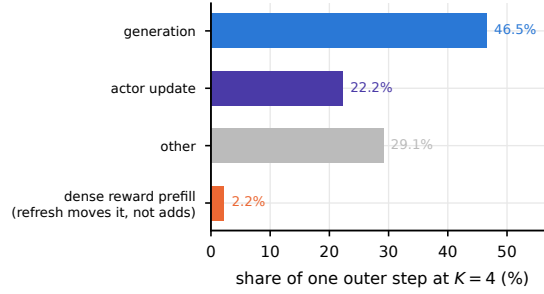


Figure 8: Composition of one outer step at $K=4$, averaged over steady training steps. The dense-reward prefill is 2.2% of a step; reward refresh changes only when and at which student parameters that prefill runs, not how many times it runs per outer step, and so leaves throughput unchanged.

2015; Bengio et al., 2015), which imitation learning addresses by querying the expert on the learner’s own state distribution (Ross et al., 2010; Lin et al., 2020).

On-policy distillation. OPD applies that correction directly: the student samples, and the teacher scores those samples token by token (Gu et al., 2023; Agarwal et al., 2023). Variants change the divergence (Wen et al., 2023), add a contrastive term (Ko et al., 2024; 2025), or alter the sampling protocol (Xu et al., 2024). Li et al. (2026) establish the phenomenology and a recipe for the single-teacher case, and a fast-growing literature now asks where the dense signal helps and where it hurts (Song & Zheng, 2026; Wang et al., 2026d; Armandpour et al., 2026; Fu et al., 2026; Zhang et al., 2026; Zhao et al., 2026a), which tokens carry it (Xu et al., 2026b; Jiang et al., 2026; Yu et al., 2026b;d), and how to stabilise or extend it: through warm-up and target reformulation (Liu et al., 2026; Jang et al., 2026; Xie et al., 2026b; Jin et al., 2026), learning past the teacher, including a teacher weaker than the student (Yang et al., 2026; Lu et al., 2026; Yu et al., 2026a; Feng et al., 2026), efficiency (Ko et al., 2026; Wu et al., 2026a;b), alternative supervision targets (Ye et al., 2026b; Fang et al., 2026; Yu et al., 2026c; Heo et al., 2026; Ma, 2026; Xu et al., 2026a; Wang et al., 2026a), continual post-training (Wang et al., 2026c), agentic and experiential settings (Pan et al., 2026; Ye et al., 2026a), and mismatched tokenizers (Sun et al., 2026; Wang et al., 2026b). All of this work is single-teacher, so the question of how several teachers share one student’s update budget does not arise.

Staleness under batch reuse. Reward refresh addresses a staleness that also appears in asynchronous RL systems, where rollouts are produced by a policy several updates behind the learner (Fu et al., 2025; Zhong et al., 2025; Sheng et al., 2025; Yan et al., 2025; Gao et al., 2025; Sheng et al., 2024). How much staleness such systems tolerate has been measured for RL (Zheng et al., 2025; Li et al., 2025a; Khatri et al., 2025) and for OPD in particular (Kang et al., 2026; Zheng & Jiang, 2026). Our setting is synchronous: the rollout batch is reused for several inner updates, so what goes stale is the reward attached to a token rather than the policy that produced it.

From one teacher to many. Distilling several teachers into one student predates LLMs, in ensemble distillation with fixed or learned teacher weights (You et al., 2017; Liu et al., 2021b; Yuan et al., 2020), and in reinforcement learning, where policy distillation compresses several task policies into a single network (Rusu et al., 2015; Parisotto et al., 2015; Teh et al., 2017; Czarnecki et al., 2019). Ma et al. (2026a) bring the construction to LLM post-training as MOPD, routing each prompt to its domain teacher and distilling on the student’s own rollouts, and report it in the post-training of a frontier model (Team et al., 2026). The construction has been taken up quickly, for multilingual speech recognition (Xie et al., 2026a), tool use (Shen et al., 2026), multi-task privileged context (Ma et al., 2026b), omni-modal students (Zhao et al., 2026b), expert-to-generalist replay (Chen et al., 2026), domain-specific experts (Li & Liu, 2026) and open model releases (Park et al., 2026).

These works report end-to-end outcomes; what remains open, and what we supply, is a reproducible recipe together with a measurement of what governs the training dynamics once several teachers write into the same parameters.

6.2 CAPABILITY INTEGRATION OF MODELS

Per-domain experts are cheap to make and hard to combine. Policy-gradient post-training (Schulman et al., 2017; Ouyang et al., 2022) and its verifiable-reward form (Shao et al., 2024; DeepSeek-AI et al., 2025a; Yu et al., 2025; Zhang et al., 2025) reliably produce a strong specialist in whatever domain the reward is defined: mathematics (Cobbe et al., 2021; Hendrycks et al., 2021; Albalak et al., 2025; He et al., 2025c; Hu et al., 2025; He et al., 2025b;a), with process supervision as an alternative signal (Lightman et al., 2023; Uesato et al., 2022); software engineering and code (Jimenez et al., 2023; Jain et al., 2025; Wei et al., 2025; Jain et al., 2024); instruction following (Zhou et al., 2023; Pyatkin et al., 2025); and search or browsing agents (Jin et al., 2025; Nakano et al., 2021). Preference-based objectives cover the domains where no verifier exists (Rafailov et al., 2023; Meng et al., 2024; Zheng et al., 2023), at the price of reward overoptimisation (Gao et al., 2022). Each pipeline yields one expert; deployment wants one model.

Integration in data space. The direct route is to train the domains together, either by pooling prompts into one RL run (Yang et al., 2025; Lambert et al., 2024; Grattafiori et al., 2024; Team et al., 2025; OpenAI et al., 2025; DeepSeek-AI et al., 2025b; 2026; GLM-5-Team et al., 2026) or by running them in sequence (Wang et al., 2025; Bercovich et al., 2025). Both inherit the difficulty that multi-task learning has documented for a decade: tasks interfere, and which ones can be trained together is not predictable from their definitions (Standley et al., 2019). The remedies operate on the gradient or the loss, by normalising task gradient magnitudes (Chen et al., 2017), projecting away conflicting components (Yu et al., 2020; Wang et al., 2020b; Liu et al., 2021a), or weighting losses by uncertainty or a multi-objective criterion (Kendall et al., 2017; Sener & Koltun, 2018). Sequential training instead faces forgetting (Kirkpatrick et al., 2016; Luo et al., 2023). A parallel line controls the mixture itself rather than the update, reweighting domains during training (Xie et al., 2023; Fan et al., 2023; Ye et al., 2024; Chen et al., 2023). Our diagnosis is in this tradition but concerns a different object: not which prompts to sample, but how much of the gradient token budget each domain receives once its answers are generated.

Integration in weight space and in module space. A second route combines finished experts without further training. Averaging weights of models fine-tuned from a shared initialisation (Wortsman et al., 2022) generalises to arithmetic on task vectors (Ilharco et al., 2022), with later work resolving sign and magnitude interference between them (Yadav et al., 2023; Yu et al., 2023; Yang et al., 2023), weighting parameters by an estimate of their importance (Matena & Raffel, 2021; Jin et al., 2022), aligning permutation symmetries (Ainsworth et al., 2022), or searching merge recipes directly (Akiba et al., 2024; Jang et al., 2024); a related line fuses heterogeneous models through their output distributions (Wan et al., 2024a;b). A third route keeps the experts separate and routes between them, by training expert branches and then merging or mixing them (Li et al., 2022; Sukhbaatar et al., 2024), by turning a dense checkpoint into a sparse mixture (Komatsuzaki et al., 2022), by conditional computation (Shazeer et al., 2017; Fedus et al., 2021; Jiang et al., 2024), or by composing lightweight adapters at inference (Huang et al., 2023; Wu et al., 2024; Muqeeth et al., 2024). Weight-space fusion needs no rollouts but is sensitive to the merge recipe, and module-space routing keeps the experts’ parameter cost at deployment. Multi-teacher OPD instead performs the integration in policy space, and this paper studies the cost that choice carries: the domains no longer interfere through weights or through a shared reward scale, but through the optimisation budget they draw from a single student.

7 CONCLUSION

We build from scratch a fully open multi-teacher on-policy distillation pipeline—mixed-domain SFT, three domain RL teachers, multi-teacher OPD—and use it to answer one

concrete question: when routing is already error-free, what prevents the capabilities of three experts from being written into the same set of parameters at once. The answer is not semantic conflict between the teachers but the allocation of optimisation budget, which shows up as three mutually independent axes of imbalance that all point at the budget. The three mechanisms of Open-MOPD—token-share balancing, gap-following allocation, reward refresh—correspond one by one to these three measurements and are validated separately on a cumulative ladder, and together they compress the integration gap from 3.50 points to 0.31 points (recovery relative to RouteRL rising from 35.6% to 83.4%). We release the complete recipe and the mechanism implementations to inspire future research within the field.

REFERENCES

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes, 2023.
- Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git Re-Basin: Merging Models modulo Permutation Symmetries, 2022.
- Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary Optimization of Model Merging Recipes, 2024.
- Alon Albalak, Duy Phung, Nathan Lile, Rafael Rafailov, Kanishk Gandhi, Louis Castri-cato, Anikait Singh, Chase Blagden, et al. Big-Math: A Large-Scale, High-Quality Math Dataset for Reinforcement Learning in Language Models, 2025.
- Mohammadreza Armandpour, Fatih Ilhan, David Harrison, Ajay Jaiswal, Duc N. M Hoang, Fartash Faghri, Yizhe Zhang, Minsik Cho, and Mehrdad Farajtabar. Unmasking On-Policy Distillation: Where It Helps, Where It Hurts, and Why, 2026.
- Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks, 2015.
- Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, et al. Llama-Nemotron: Efficient Reasoning Models, 2025.
- Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation Scaling Laws, 2025.
- Mayee F. Chen, Nicholas Roberts, Kush Bhatia, Jue Wang, Ce Zhang, Frederic Sala, and Christopher Ré. Skill-it! A Data-Driven Skills Framework for Understanding and Training Language Models, 2023.
- Yunjie Chen, Xiaoxin Chen, and Fang Wang. REGEN: Replay-recycling for Expert-to-Generalist distillation with Offline Reinforcement Learning, 2026.
- Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks, 2017.
- Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation, 2019.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, et al. Training Verifiers to Solve Math Word Problems, 2021.
- Wojciech Marian Czarnecki, Razvan Pascanu, Simon Osindero, Siddhant M. Jayakumar, Grzegorz Swirszcz, and Max Jaderberg. Distilling Policy Distillation, 2019.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, 2025a.
- DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, et al. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models, 2025b.
- DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, et al. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence, 2026.
- Simin Fan, Matteo Pagliardini, and Martin Jaggi. DoGE: Domain Reweighting with Generalization Estimation, 2023.

- Junfeng Fang, Zhepei Hong, Mao Zheng, Mingyang Song, Gengsheng Li, Houcheng Jiang, Dan Zhang, Haiyun Guo, Xiang Wang, and Tat-Seng Chua. Rubric-based On-policy Distillation, 2026.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity, 2021.
- Shiyuan Feng, Huan-ang Gao, Haohan Chi, Hanlin Wu, Zhilong Zhang, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Weak-to-Strong Generalization via Direct On-Policy Distillation, 2026.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, et al. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning, 2025.
- Yuqian Fu, Haohuan Huang, Kaiwen Jiang, Jiakai Liu, Zhuo Jiang, Yuanheng Zhu, and Dongbin Zhao. Revisiting On-Policy Distillation: Empirical Failure Modes and Simple Fixes, 2026.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling Laws for Reward Model Overoptimization, 2022.
- Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, et al. RollPacker: Mitigating Long-Tail Rollouts for Fast, Synchronous RL Post-Training, 2025.
- GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, et al. GLM-5: from Vibe Coding to Agentic Engineering, 2026.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, et al. The Llama 3 Herd of Models, 2024.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: On-Policy Distillation of Large Language Models, 2023.
- Bingxiang He, Zekai Qu, Zeyuan Liu, Yinghao Chen, Yuxin Zuo, Cheng Qian, Kaiyan Zhang, Weize Chen, et al. JustRL: Scaling a 1.5B LLM with a Simple RL Recipe, 2025a.
- Jujie He, Jiakai Liu, Chris Yuhao Liu, Rui Yan, Chaojie Wang, Peng Cheng, Xiaoyu Zhang, Fuxiang Zhang, et al. Skywork Open Reasoner 1 Technical Report, 2025b.
- Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, et al. DeepMath-103K: A Large-Scale, Challenging, Decontaminated, and Verifiable Mathematical Dataset for Advancing Reasoning, 2025c.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset, 2021.
- Byeongho Heo, Jaehui Hwang, Sangdoo Yun, and Dongyoon Han. On-Policy Delta Distillation, 2026.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network, 2015.
- Namgyu Ho, Laura Schmid, and Se-Young Yun. Large Language Models Are Reasoning Teachers, 2022.
- Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes, 2023.

- Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-Reasoner-Zero: An Open Source Approach to Scaling Up Reinforcement Learning on the Base Model, 2025.
- Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. LoRaHub: Efficient Cross-Task Generalization via Dynamic LoRA Composition, 2023.
- Hugging Face. SmolLM3: Smol, multilingual, long-context reasoner. <https://huggingface.co/blog/smollm3>, 2025.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing Models with Task Arithmetic, 2022.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024.
- Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents, 2025.
- Dong-Hwan Jang, Sangdoo Yun, and Dongyoon Han. Model Stock: All we need is just a few fine-tuned models, 2024.
- Ijun Jang, Jewon Yeom, Juan Yeo, Hyunggyu Lim, and Taesup Kim. Stable On-Policy Distillation through Adaptive Target Reformulation, 2026.
- Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, et al. Mixtral of Experts, 2024.
- Yuxuan Jiang, Runchao Li, Shubhashis Roy Dipta, Dawei Li, and Zhao Yang. Cornerstones or Stumbling Blocks? Deciphering the Rock Tokens in On-Policy Distillation, 2026.
- Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for Natural Language Understanding, 2019.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, 2023.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning, 2025.
- Woogyol Jin, Taywon Min, Yongjin Yang, Dennis Wei, Yi Zhou, Swanand Ravindra Kadhe, Nathalie Baracaldo, and Kimin Lee. Entropy-Aware On-Policy Distillation of Language Models, 2026.
- Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless Knowledge Fusion by Merging Weights of Language Models, 2022.
- Wonjun Kang, Kevin Galim, Seunghyuk Oh, Minjun Kang, Sanghyun Park, Donghoon Kim, Minjae Lee, Minseo Kim, et al. AsyncOPD: How Stale Can On-Policy Distillation Be?, 2026.
- Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics, 2017.
- Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The Art of Scaling Reinforcement Learning Compute for LLMs, 2025.

- Yoon Kim and Alexander M. Rush. Sequence-Level Knowledge Distillation, 2016.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, et al. Overcoming catastrophic forgetting in neural networks, 2016.
- Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. DistiLLM: Towards Streamlined Distillation for Large Language Models, 2024.
- Jongwoo Ko, Tianyi Chen, Sungnyun Kim, Tianyu Ding, Luming Liang, Ilya Zharkov, and Se-Young Yun. DistiLLM-2: A Contrastive Approach Boosts the Distillation of LLMs, 2025.
- Jongwoo Ko, Sara Abdali, Young Jin Kim, Tianyi Chen, and Pashmina Cameron. Scaling Reasoning Efficiently via Relaxed On-Policy Distillation, 2026.
- Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Yi Tay, Mostafa Dehghani, and Neil Houlsby. Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints, 2022.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training, 2024.
- Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tim Althoff, Noah A. Smith, and Luke Zettlemoyer. Branch-Train-Merge: Embarrassingly Parallel Training of Expert Language Models, 2022.
- Xiaocan Li, Shiliang Wu, and Zheng Shen. A-3PO: Accelerating Asynchronous LLM Training with Staleness-aware Proximal Policy Approximation, 2025a.
- Xuefeng Li and Pengfei Liu. UMOE:Unlocking Every Expert in Domain-Specific Training, 2026.
- Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, et al. Rethinking On-Policy Distillation of Large Language Models: Phenomenology, Mechanism, and Recipe, 2026.
- Yuetai Li, Xiang Yue, Zhangchen Xu, Fengqing Jiang, Luyao Niu, Bill Yuchen Lin, Bhaskar Ramasubramanian, and Radha Poovendran. Small Models Struggle to Learn from Strong Reasoners, 2025b.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step, 2023.
- Alexander Lin, Jeremy Wohlwend, Howard Chen, and Tao Lei. Autoregressive Knowledge Distillation through Imitation Learning, 2020.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-Averse Gradient Descent for Multi-task Learning, 2021a.
- Tao Liu, Taiqiang Wu, Mao Zheng, Xuan Luo, Runming Yang, Xuewei Yang, Junjie Wang, and Yujiu Yang. Simple-OPD: Demystifying Warm-up for On-policy Distillation, 2026.
- Yuang Liu, Wei Zhang, and Jun Wang. Adaptive Multi-Teacher Multi-level Knowledge Distillation, 2021b.
- Songshuo Lu, Zhi Chen, and Yaohua Tang. Beyond the Best Teacher: Expanding and Compressing the Reasoning Solution Manifold, 2026.
- Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning, 2023.

- Guoqing Ma. Outcome-Confounded Local Supervision in On-Policy Distillation, 2026.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, et al. MOPD: Multi-Teacher On-Policy Distillation for Capability Integration in LLM Post-Training, 2026a.
- Yingzi Ma, Zichen Zhu, Ming Jiang, and Chaowei Xiao. One Student, Many Teachers: Multi-Task On-Policy Distillation via Soft-Prompt Privileged Context, 2026b.
- Michael Matena and Colin Raffel. Merging Models with Fisher-Weighted Averaging, 2021.
- Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple Preference Optimization with a Reference-Free Reward, 2024.
- Seyed-Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, Nir Levine, Akihiro Matsukawa, and Hassan Ghasemzadeh. Improved Knowledge Distillation via Teacher Assistant, 2019.
- Mohammed Muqeeth, Haokun Liu, Yufan Liu, and Colin Raffel. Learning to Route Among Specialized Experts for Zero-Shot Generalization, 2024.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, et al. WebGPT: Browser-assisted question-answering with human feedback, 2021.
- OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, et al. gpt-oss-120b & gpt-oss-20b Model Card, 2025.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, et al. Training language models to follow instructions with human feedback, 2022.
- Tianjun Pan, Yuan Li, Hongda Wang, Linbo Jin, Mengfei Song, Lei Gao, Qiming Shi, Shaokang Fu, et al. Bidirectional Context Self-Distillation for Reinforcement Learning of Skill-Based LLM Agents, 2026.
- Emilio Parisotto, Jimmy Lei Ba, and Ruslan Salakhutdinov. Actor-Mimic: Deep Multitask and Transfer Reinforcement Learning, 2015.
- Sungrae Park, Sanghoon Kim, Gyoungjin Gim, Jungho Cho, Hyunwoong Ko, Minbyul Jeong, Minjeong Kim, Keunwoo Choi, et al. Solar Open 2 Technical Report, 2026.
- Hao Peng, Xin Lv, Yushi Bai, Zijun Yao, Jiajie Zhang, Lei Hou, and Juanzi Li. Pre-training Distillation for Large Language Models: A Design Space Exploration, 2024.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing Verifiable Instruction Following, 2025.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model, 2023.
- Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence Level Training with Recurrent Neural Networks, 2015.
- Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, 2010.
- Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. Policy Distillation, 2015.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019.

- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, 2017.
- Ozan Sener and Vladlen Koltun. Multi-Task Learning as Multi-Objective Optimization, 2018.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer, 2017.
- Jiabin Shen, Guang Chen, and Chengjun Mao. When Top-K Misses the Decision: Tool-Call Drift in Multi-Teacher On-Policy Distillation, 2026.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A Flexible and Efficient RLHF Framework, 2024.
- Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, et al. Laminar: A Scalable Asynchronous RL Post-Training Framework, 2025.
- Mingyang Song and Mao Zheng. A Survey of On-Policy Distillation for Large Language Models, 2026.
- Trevor Standley, Amir R. Zamir, Dawn Chen, Leonidas Guibas, Jitendra Malik, and Silvio Savarese. Which Tasks Should Be Learned Together in Multi-task Learning?, 2019.
- Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Daniel Li, et al. Branch-Train-MiX: Mixing Expert LLMs into a Mixture-of-Experts LLM, 2024.
- Jie Sun, Mao Zheng, Mingyang Song, Qiyong Zhong, Yilin Cheng, Bichuan Feng, Pengfei Liu, Junfeng Fang, and Xiang Wang. SimCT: Recovering Lost Supervision for Cross-Tokenizer On-Policy Distillation, 2026.
- Core Team, Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, et al. MiMo-V2-Flash Technical Report, 2026.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, et al. Kimi k1.5: Scaling Reinforcement Learning with LLMs, 2025.
- Yee Whye Teh, Victor Bapst, Wojciech Marian Czarnecki, John Quan, James Kirkpatrick, Raia Hadsell, Nicolas Heess, and Razvan Pascanu. Distral: Robust Multitask Reinforcement Learning, 2017.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022.
- Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge Fusion of Large Language Models, 2024a.
- Fanqi Wan, Longguang Zhong, Ziyi Yang, Ruijun Chen, and Xiaojun Quan. FuseChat: Knowledge Fusion of Chat Models, 2024b.
- Boxin Wang, Chankyu Lee, Nayeon Lee, Sheng-Chieh Lin, Wenliang Dai, Yang Chen, Yangyi Chen, Zhuolin Yang, et al. Nemotron-Cascade: Scaling Cascaded Reinforcement Learning for General-Purpose Reasoning Models, 2025.
- Chen Wang, Zhaochun Li, Jionghao Bai, Yining Zhang, Hexuan Deng, Ge Lan, and Yue Wang. Distilled Reinforcement Learning for LLM Post-training, 2026a.

- Hao Wang, Kun Yuan, Wenlin Zhong, Minglei Zhang, Han Xiao, Ming Sun, and Honggang Qi. Cross-Tokenizer On-Policy Distillation via Byte-Prefix Marginalization, 2026b.
- Meng Wang, Haohan Zhao, Wenzhuo Liu, Lu Yang, Geng Liu, Haiyang Guo, Guo-Sen Xie, Gaofeng Meng, Hongbin Liu, and Fei Zhu. Denser $\neq$ Better: Limits of On-Policy Self-Distillation for Continual Post-Training, 2026c.
- Rui Wang, Hongru Wang, Yi Chen, Boyang Xue, Tianqing Fang, Wenhao Yu, and Kam-Fai Wong. Demystifying On-Policy Distillation: Roles, Pathologies, and Regulations, 2026d.
- Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers, 2020a.
- Zirui Wang, Yulia Tsvetkov, Orhan Firat, and Yuan Cao. Gradient Vaccine: Investigating and Improving Multi-task Optimization in Massively Multilingual Models, 2020b.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution, 2025.
- Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. f-Divergence Minimization for Sequence-Level Knowledge Distillation, 2023.
- Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time, 2022.
- Xun Wu, Shaohan Huang, and Furu Wei. Mixture of LoRA Experts, 2024.
- Yecheng Wu, Song Han, and Hai Cai. Lightning OPD: Efficient Post-Training for Large Reasoning Models with Offline On-Policy Distillation, 2026a.
- Yecheng Wu, Song Han, and Han Cai. Lightning OPD 2.0: Mitigating Style Bias in Cross-Teacher On-Policy Distillation for Large Reasoning Models, 2026b.
- Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining, 2023.
- Yuan Xie, Jiaqi Song, Xianliang Wang, Ming Lei, Jie Gao, and Jie Wu. Language-Specialized Multi-Teacher On-Policy Distillation for Multilingual LLM-Based ASR, 2026a.
- Zhengpeng Xie, Li Lyna Zhang, Zeke Xie, and Mao Yang. Trust Region Policy Distillation, 2026b.
- Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. Speculative Knowledge Distillation: Bridging the Teacher-Student Gap Through Interleaved Sampling, 2024.
- Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, Zhipeng Wang, and Alborz Geramifard. Beyond GRPO and On-Policy Distillation: An Empirical Sparse-to-Dense Reward Principle for Language-Model Post-Training, 2026a.
- Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, Zhipeng Wang, and Alborz Geramifard. TIP: Token Importance in On-Policy Distillation, 2026b.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving Interference When Merging Models, 2023.
- Ran Yan, Youhe Jiang, Tianyuan Wu, Jiakuan Gao, Zhiyu Mei, Wei Fu, Haohui Mai, Wei Wang, Yi Wu, and Binhang Yuan. AReaL-Hex: Accommodating Asynchronous RL Training over Heterogeneous GPUs, 2025.

- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, et al. Qwen3 Technical Report, 2025.
- Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive Model Merging for Multi-Task Learning, 2023.
- Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond Teacher: Generalized On-Policy Distillation with Reward Extrapolation, 2026.
- Jiasheng Ye, Peiju Liu, Tianxiang Sun, Jun Zhan, Yunhua Zhou, and Xipeng Qiu. Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance, 2024.
- Tianzhu Ye, Li Dong, Guanheng Chen, He Zhu, Xun Wu, Shaohan Huang, and Furu Wei. LLM-as-a-Coach: Experiential Learning for Non-Verifiable Tasks, 2026a.
- Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-Policy Context Distillation for Language Models, 2026b.
- Shan You, Chang Xu, Chao Xu, and Dacheng Tao. Learning from multiple teacher networks. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2017.
- Fangxu Yu, Weijia Xu, Michael Xu, Tianyi Zhou, and Zinan Lin. Weak-to-Strong On-Policy Distillation, 2026a.
- Guo Yu, Wenlin Liu, Yulan Hu, Hao-Xuan Ma, Jun-Peng Jiang, and Han-Jia Ye. Dense Supervision, Sparse Updates: On the Sparsity and Geometry of On-Policy Distillation, 2026b.
- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale, 2025.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning, 2020.
- Weichen Yu, Xiaomin Li, Yizhou Zhao, Xiaoze Liu, Ruowang Zhang, Haixin Wang, Yinyi Luo, Chen Henry Wu, et al. Multi-Rollout On-Policy Distillation via Peer Successes and Failures, 2026c.
- Zichao Yu, Chengzhi Yu, Shengze Xu, Yujin Han, Bingqing Jiang, Xu Wang, and Difan Zou. Mismatch Matters: On-Policy Distillation Beyond Token Agreement, 2026d.
- Fei Yuan, Linjun Shou, Jian Pei, Wutao Lin, Ming Gong, Yan Fu, and Daxin Jiang. Reinforced Multi-Teacher Selection for Knowledge Distillation, 2020.
- Huaqing Zhang, Jingchu Gai, Juno Kim, Bingbin Liu, and Andrej Risteski. When Does Online Imitation Learning Help in LLM Post-Training? The Role of (Non-)Realizability Beyond Horizon, 2026.
- Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, et al. A Survey of Reinforcement Learning for Large Reasoning Models, 2025.
- Anhao Zhao, Haoran Xin, Yingqi Fan, Junlong Tong, Wenjie Li, and Xiaoyu Shen. Decoupling KL and Trajectories: A Unified Perspective for SFT, DAGger, Offline RL, and OPD in LLM Distillation, 2026a.
- Tong Zhao, Yuyang Hu, Yutao Zhu, Reed Li, Haijin Liang, Haibo Shi, Yu Lu, and Zhicheng Dou. OPOD: On-Policy Omni Distillation, 2026b.

- Haizhong Zheng, Jiawei Zhao, and Beidi Chen. Prosperity before Collapse: How Far Can Off-Policy RL Reach with Stale Data on LLMs?, 2025.
- Liwen Zheng and Haiyun Jiang. Blockwise Policy-Drift Gating for On-Policy Distillation, 2026.
- Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, et al. Secrets of RLHF in Large Language Models Part I: PPO, 2023.
- Yinmin Zhong, Zili Zhang, Xiaoniu Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, et al. StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated Stream Generation, 2025.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-Following Evaluation for Large Language Models, 2023.

Table 5: Base-model screening under the same OpenR1-Math-93k SFT. Each cell is AIME24 accuracy / generation truncation rate (%). Qwen3-1.7B-Base mainly generates long, unclosed reasoning trajectories at every observed checkpoint; Qwen2.5-7B-Base instead raises accuracy and lowers truncation together as SFT proceeds.

Epoch	Step	Qwen3-1.7B-Base	Qwen2.5-7B-Base
1	732	4.58 / 75.42	16.25 / 42.92
2	1464	7.08 / 72.50	20.83 / 35.42
3	2196	6.67 / 80.42	30.83 / 22.08
4	2928	5.83 / 69.17	30.42 / 19.58
5	3660	6.67 / 69.58	31.25 / 12.92

A CONSTRUCTING THE OPEN-MOPD RECIPE

The final Open-MOPD recipe was not obtained directly from a single model and a single data mixture. We first screen on math SFT for a base model that stably produces reasoning trajectories, and then extend the selected base model to a mixed-domain SFT over math, code and instruction following. This section records that construction process, so that the model choice in Section 2.2 can be traced back to concrete experiments rather than being decided by parameter-count experience.

A.1 BASE MODEL CHOICE

OpenR1 math comparison between 1.7B and 7B. Early screening applied the same 93,733 OpenR1 math examples to Qwen3-1.7B-Base and Qwen2.5-7B-Base separately. Both runs use a 32,768-token SFT length limit, a global batch of 128 and a learning rate of 4×10^{-5} , with 732 steps per epoch. Evaluation is AIME24, sampling each of the 30 problems 8 times for 240 rollouts in total; the generation limit is 31,000 tokens. Table 5 reports the complete curve over the first five epochs.

The problem with Qwen3-1.7B-Base is not a lack of generation length: under the 31K budget, 69.17–80.42% of the samples still exhaust the limit at all five checkpoints, and only 20.00–31.25% of the answers close `</think>`; adding epochs does not produce a monotone improvement either, and AIME24 stays below 7.1%. This means that a large fraction of the trajectories seen by a subsequent RL or OPD stage contain only a reasoning prefix and never reach a final answer. A drop in task score can then not be attributed uniquely to the reward, the routing or cross-domain interference, because the base model and the data mixture themselves have not yet learnt to end a reasoning trajectory reliably.

The same OpenR1 data shows the opposite trend on Qwen2.5-7B-Base: AIME24 rises from 16.25% to 31.25% and the truncation rate falls from 42.92% to 12.92%. This rules out the explanation that “OpenR1 trajectories are inherently too long, so any base model would truncate”, and it shows that enlarging the capacity of the base model does recover trajectory closure. Qwen2.5-7B-Base therefore satisfies the capability condition, but it puts the student, the three RL teachers and every mechanism ablation of the full recipe at 7B scale; for an open research baseline that has to be run repeatedly, this cost is too high.

Choosing SmolLM3-3B-Base. The final recipe selects SmolLM3-3B-Base as the compromise between the two ends. Four epochs of mixed-domain SFT over the three domains give π_{mixsft} : OpenR1-Math-93k, the OCR-50k subset sampled from the full OpenCodeReasoning set, and Instruction-Nemotron contribute roughly 37.3%, 28.1% and 34.6% of the training response tokens respectively. The per-domain sample counts are balanced by response token count, so that the 820K short IF answers do not drown out the fewer but longer math and code trajectories.

Table 6 summarises the selection evidence for the three candidate base models. Under the unified six-benchmark validation, π_{mixsft} from SmolLM3-3B-Base obtains math 16.37, IF 41.84 and code 16.65, for a total of 24.95; with the math OPD response cap set to 16,384, the measured training truncation rate is 15.2% at a mean response length of 11,422, showing

Table 6: Base-model choice for Open-MOPD. The table summarises directly the experimental results used when constructing the recipe; the truncation evidence notes the corresponding stage.

Base model	SFT data	Performance	Truncation evidence and conclusion
Qwen3-1.7B-Base	OpenR1-Math-93k	AIME24 best 7.08	Truncation 69.17–80.42% under a 31K generation budget; capacity and trajectory closure insufficient.
SmolLM3-3B-Base	Three-domain mixed-domain SFT $\rightarrow \pi_{\text{mixsft}}$	math/IF/code = 16.37/41.84/16.65; total 24.95	Math OPD truncation 15.2% under a 16K limit; chosen as the base model of the open recipe.
Qwen2.5-7B-Base	OpenR1-Math-93k	AIME24 31.25	Truncation 12.92% under a 31K generation budget; capable enough, but a full multi-teacher ablation is too costly.

that the bulk of long reasoning enters dense supervision intact. This base model avoids the trajectory-closure failure of 1.7B while keeping the subsequent student and three-teacher experiments at a scale that can be ablated repeatedly.

3B is therefore not an arbitrary choice of a “medium model” but an operating point bracketed by two failure boundaries: going down to 1.7B, generation is dominated by unclosed reasoning and truncation, and mechanism attribution is no longer unique; going up to 7B, the single model is stronger, but the end-to-end multi-teacher recipe and its repeated ablations exceed the resource boundary we set for a community baseline. SmolLM3-3B-Base retains enough trajectory capacity while still allowing the final Open-MOPD training to run on a single 8×A100-80GB node.

A.2 STAGE HYPERPARAMETERS

This section lists all hyperparameters of the four stages of the recipe, taken from the training configurations that were actually run rather than from recommended values compiled after the fact. Table 7 gives the mixed-domain SFT and the three domain RL teachers, and Table 8 gives single-domain OPD (the RouteOPD oracle of the main text) and multi-teacher OPD. Items not listed use the verl defaults.

Mixed-domain SFT and domain RL teachers. The three teachers share one GRPO configuration and differ only in data, sequence length, rollout group size and number of steps: IF answers are short, so it uses a smaller response limit, smaller groups and more steps; math and code use a 30,000-token response limit to accommodate the full reasoning chain. All three disable the KL penalty and apply group filtering on accuracy (the samples of one prompt are discarded if they are all correct or all wrong, with at most 8 generation batches resampled), so that the gradient comes only from groups that discriminate.

Single-domain OPD and multi-teacher OPD. The two share the same optimiser and distillation settings, so the difference between the RouteOPD oracle and M-OPD in the main text comes only from the number of teachers and the domain mixture, not from the training configuration. The dense reward is computed on the student top- k ($k=16$), and $K=\text{train batch}/\text{mini batch}=4$ is the off-policy depth. The multi-teacher column additionally enables the domain sampler and the three mechanism switches: token-share balancing sets the target gradient share, gap-following allocation adjusts the domain weights by the

Table 7: Hyperparameters of stages one and two. SFT is standard supervised fine-tuning; the three RL teachers are all GRPO, each initialised from π_{mixsft} and trained only on the verifiable reward of its own domain. World size is given as number of nodes $\times$ GPUs per node, all A100-80GB. “—” means the item does not apply to that stage.

	Mixed-domain SFT	RL: math	RL: code	RL: IF
World size	8 $\times$ 8	8 $\times$ 8	8 $\times$ 8	4 $\times$ 8
Starting point	SmolLM3-3B-Base		π_{mixsft}	
Algorithm	Cross-entropy		GRPO	
train batch size	128	128	128	128
mini batch size	—	32	32	32
Learning rate	4×10^{-5}		1×10^{-6}	
lr schedule	cosine, warmup 3%	constant, warmup 10 steps		
grad clip	0.2		1.0	
Prompt limit	—	1,024	2,048	2,048
Response limit	32,768 (total length)	30,000	30,000	2,048
rollout n	—	16	16	8
Temperature	—		1.0	
epochs / steps	4 epochs	1,000 steps	1,000 steps	3,000 steps
clip _{low} /clip _{high}	—		0.2 / 0.25	
KL penalty	—		disabled	
entropy coeff	—		0	
group filtering	—	by acc, at most 8 generation batches		

Table 8: Hyperparameters of stage three. The left columns are the three single-domain OPD runs (the RouteOPD oracle of the main text) and the right column is multi-teacher OPD. The two share the optimiser, sequence lengths and distillation settings; the token-share balancing / gap-following allocation / reward refresh rows are the values of our mechanisms in the final recipe.

	Single-domain OPD (RouteOPD oracle)			Multi-teacher OPD
	math	code	IF	Open-MOPD
World size	4 $\times$ 8	4 $\times$ 8	1 $\times$ 8	4 $\times$ 8
Teacher	$\pi_{\phi_{\text{math}}}$	$\pi_{\phi_{\text{code}}}$	$\pi_{\phi_{\text{IF}}}$	3-teacher hard routing
Student starting point				π_{mixsft}
train batch size				1,024
mini batch size				256 (i.e. $K=4$)
Learning rate				1.5×10^{-6} , constant
clip _{low} /clip _{high}				0.2 / 0.28
KL penalty				disabled
Advantage estimation				dense reward written directly into the advantage slot
Prompt limit	1,024	2,048	2,048	2,048
Response limit	16,384	16,384	1,024	16K (math/code) / 2K (IF)
Total steps			600 (validate and checkpoint every 25 steps)	
Reward form	single-teacher dense reward			multi-teacher hard-routed dense reward
top- k	16, from the student distribution, nucleus truncation $p=0.99$			
Reward weighting	weighted by student probability $\tilde{\pi}_{\theta}$ (Eq. 3)			
Domain sampling	—	—	—	weighted sampling, math:code:IF = 2:2:1
share target g^*	—	—	—	(1/3, 1/3, 1/3)
gap-following α	—	—	—	1.0 (m_d/m_{ref} forward)
reward refresh			enabled	enabled

remaining gap, and reward refresh refreshes the dense reward within every inner update; the table gives their values in the final recipe.

The form of instability discussed in Section 5.2 comes from the reverse branch of gap-following allocation: it introduces no reference quantity m_{ref} and applies no clipping to the weights, so a positive α keeps pushing the budget towards the domain with the smallest m_d , that is, the domain already closest to its teacher. The two seeds of the ablation are configured identically apart from the random seed.